

The failures flashed in the above run are due to a quick testing criterion of running `sshd` in containers, which is not true in the case of our image. Our image is not a continuously running service and that's why we gave it a sleep command. This way the launched service container is present during the `dgoss` operations of copying `goss` and `goss.yaml` into it and launching a `goss` test run in it. The `dgoss` behaviour is configurable by a number of environment variables, which can be seen in its documentation link provided in the References section.

Now let's run a more standard example using a Docker image of a very popular high-performance engine for large-scale computing tasks, known as Apache Spark. First, pull the official Spark Docker image using the command `docker pull bitnami/spark:latest`. Next, prepare a `goss` tests configuration file `spark.yaml` in your current working directory, as shown below:

```
--- spark.yaml ---
```

```
port:
  tcp:7077:
    listening: true
  tcp6:7077:
    listening: true
  tcp:8080:
    listening: true
```

```
INFO: Creating docker container
INFO: Copy goss files into container
Successfully copied 13.7MB to 1b1bdde5eaac0c301bfda6701665dc7ac967a4e00ebb0f894a0ebc124bd666df:/goss
INFO: Starting docker container
INFO: Container ID: 1b1bdde5
INFO: Sleeping for 0.2
INFO: Container health
INFO: Running Tests
1..5
ok 1 - Process: java: running: matches expectation: true
not ok 2 - Port: tcp:7077: listening: Expected false to equal true
not ok 3 - Port: tcp:8080: listening: Expected false to equal true
not ok 4 - Port: tcp6:7077: listening: Expected false to equal true
not ok 5 - Port: tcp6:8080: listening: Expected false to equal true
INFO: Deleting container
```

Figure 4: Dgoss tests that failed for Spark

```
INFO: Creating docker container
INFO: Copy goss files into container
Successfully copied 13.7MB to 3a6496fd5ed45f3566051dc3c1751a7112c573d647228cfb0e4649b8389f9206:/goss
INFO: Starting docker container
INFO: Container ID: 3a6496fd
INFO: Sleeping for 10
INFO: Container health
INFO: Running Tests
1..5
ok 1 - Process: java: running: matches expectation: true
not ok 2 - Port: tcp:7077: listening: Expected false to equal true
ok 3 - Port: tcp6:7077: listening: matches expectation: true
not ok 4 - Port: tcp:8080: listening: Expected false to equal true
ok 5 - Port: tcp6:8080: listening: matches expectation: true
INFO: Deleting container
```

Figure 5: Dgoss tests that passed for Spark

```
tcp6:8080:
  listening: true
process:
  java:
    running: true
```

Now, run the command `docker run -it --rm -v ${PWD}:/etc/goss:ro -v /var/run/docker.sock:/var/run/docker.sock:ro -e GOSS_FILE=spark.yaml -e GOSS_FILES_STRATEGY=cp -e GOSS_OPTS='--color -format tap' dgoss:0.4.9 dgoss run -e SPARK_MODE=master bitnami/spark` to launch a test run against the official Spark image. It's a bit surprising that although the Spark Java service is running, the master and UI port tests are failing, as shown in Figure 4.

Let's try once again by executing the command `docker run -it --rm -v ${PWD}:/etc/goss:ro -v /var/run/docker.sock:/var/run/docker.sock:ro -e GOSS_SLEEP=10 -e GOSS_FILE=spark.yaml -e GOSS_FILES_STRATEGY=cp -e GOSS_OPTS='--color -format tap' dgoss:0.4.9 dgoss run -e SPARK_MODE=master bitnami/spark`. Now things look better. If you have a Dockerized service, which has startup delays to run and respond to (e.g., Java/Ruby, etc), then `GOSS_SLEEP` helps to wait before launching a `goss` run in the spawned service container. All the IPv6 port tests are passing now, as shown in Figure 5.